

Self federated Anomaly Detection System For Cloud Infrastructure

SAMEER YELE¹, RUSHIKESH SAWANT¹, PRATIK BOTRE¹

¹Department of Computer Engineering, Ramrao Adik Institute of Technology, Sector 7, Nerul, Navi Mumbai

Corresponding author: Sameer Yele.

ABSTRACT Telemetry data produced by cloud computing infrastructure can amount to vast quantities of data emanating from numerous distributed computing resources, and therefore, making it impossible to monitor and detect anomalies through manual intervention. Conventional alerting systems using static thresholds are characterized by a high rate of false positives and fail to detect new attacks. In this research paper, we propose an efficient self-learning anomaly detection framework using distributed machine learning approaches together with remediation mechanisms for cloud native environment. Our approach uses a combination of Isolation Forests in metric based anomaly detection alongside Autoencoder neural networks in pattern detection based on logs. Both algorithms use a weighting mechanism to fuse their predictions and make accurate detections. Our framework makes use of federated learning approach which ensures privacy preservation during the model training process. Another important characteristic of the implemented solution is the implementation of zero-trust security architecture called “Cyber Jail” that automatically isolates the device showing signs of highly severe malicious activity. This solution enables conducting predictions concerning the expected lifetime of the critical resources based on linear regression analysis. Dashboard developed based on the Streamlit and Plotly frameworks provides monitoring capabilities as well as enables the observability of the system by generating dynamic graphs of metrics, displaying the 3D map, and creating the post-mortem report on incidents. The effectiveness of anomaly detection algorithms tested on artificially generated cloud computing telemetry showed that accuracy of such algorithms can reach 92% and above. Moreover, the proposed platform provides an opportunity to significantly reduce the value of MTTD, reaching just a few seconds, as well as allows implementing automated response options, thereby reducing the burden on SRE teams.

KEYWORDS Anomaly Detection; Cloud Computing; Machine Learning; Federated Learning; Isolation Forest; Autoencoder; Zero-Trust Security; Self-Healing Systems; eBPF; Kubernetes.

I. INTRODUCTION

THE advent of cloud computing technology has completely transformed how companies have traditionally managed and leveraged their computing resources. From reports within industry, 90 percent of companies operate using the multi-cloud or hybrid-cloud approach, meaning that thousands of microservices, virtual machines, and serverless functions are being deployed.

The current scenario within the cloud environment is one where large amounts of telemetry exist, amounting to millions of events occurring per second. Every system gathers metrics concerning CPU usage, memory utilization,

disk I/O operations, network operations, as well as logs both structured and unstructured. There could be valuable data within the enormous amounts of telemetry regarding possible failures, degraded performance, or even security risks. However, filtering out valuable data from useless data is no mean feat.

The conventional approach uses the technique of thresholding with static alerts. For instance, an alert is generated if the CPU usage exceeds 85% for five consecutive minutes. The drawback of this simple approach is the number of problems it causes. On the one hand, legitimate loads may exceed the set limit, leading to false alerts. On the other

hand, some malicious activity will go unnoticed since it does not trigger any alert due to the lack of crossing the defined threshold. Moreover, threshold setting requires certain knowledge of workloads.

An alternative approach to solving the issue is using machine learning techniques. Unlike traditional approaches that employ fixed thresholds, machine learning makes it possible to detect abnormal behavior by analyzing the typical pattern of workload operation. The primary benefit of machine learning is its ability to perform unsupervised learning. Anomalies cannot be identified due to constant change in attack patterns in cloud computing.

Nevertheless, there are several drawbacks to using a centralized approach for ML in cloud computing. Firstly, it is not possible to share any operational data among various clients due to the confidentiality policy and high degree of competitiveness. Moreover, more work is needed for handling all incoming data through telemetry, thus causing a bottleneck and single point of failure. Finally, the ML algorithm becomes obsolete, as updates take place in the system. Therefore, a decentralized solution for anomaly detection based on ML can be proposed.

A. PROBLEM STATEMENT

Research problem solved: The major research problem addressed in this study is: What are the key aspects in creating a system which would identify anomalies in a cloud infrastructure in real time, be capable of implementing preventive countermeasures and ensure complete observability, all while minimizing the expenses on it?

B. PROJECT OBJECTIVES

Some important objectives of the proposed research project include the following:

- Developing the detection technique that will leverage hybrid machine learning algorithms, where isolation forest will be leveraged for metric anomaly detection, and autoencoder neural networks will detect patterns from logs based on decision fusion approach.
- Implementing the privacy-preserving security architecture for designing the ML models based on federated learning techniques that allow training the models on telemetry data without aggregating any sensitive data.
- Development of advanced Zero Trust Isolation Technology called Cyber Jail, where devices with threats can be isolated through automated processes depending on the results of threat level detection.
- Designing an advanced real-time multi-view dashboard.
- Automate operations which will serve as remedies for particular root causes, such as those that cause rerouting of traffic, restarting of services, redirecting input/output, or constraints on resources.

II. LITERATURE REVIEW

The literature review of the existing studies forms the foundation upon which the contributions of our research work shall be done. It is clear from our analysis that although federated learning, eBPF-based observability, hybrid machine learning, and automated remediation are individually discussed in the literature, there exists no such platform that combines all these technologies. The contribution of our research work is filling the gap by designing an integrated self-learning system that works on a zero-trust security model.

III. SYSTEM ARCHITECTURE AND DESIGN

A. HIGH-LEVEL ARCHITECTURE

The proposed system architecture is based on the client-server architecture like most cloud systems.

Edge Client Layer: The cloud infrastructure involves virtual machines, containers, and bare metal servers. Telemetry agents for every edge client will handle metrics collecting, and also sending of telemetry information to the inference engine. Simulation modules for the edge client layer will include the live simulator module and injector UI module.

Tunnel Layer (Optional): This component is used to ease the connection between two network layers through ngrok tunnels. In case where inference servers and the edge clients are on separate networks, ngrok tunnels facilitate establishment of HTTPS tunneling without need for complex systems such as the VPN.

Inference Engine Layer: It is the brain of the system that was built on FastAPI framework. The activities performed by the inference engine include telemetry processing, ML predictions calculation, enforcement of security policies, persistence operation, and remediation operations.

Observability Layer: This is the command center console that is developed using Streamlit library and inference engine API data.

B. DATA FLOW AND PROCESSING PIPELINE

The process of data flow through the system includes stages from data creation till data visualization and feedback.

First of all, the telemetry data are collected from the edge devices. Metrics about CPU, RAM, disk and network activity of the system are collected periodically. Moreover, the system collects the logs that include any events, processes, and activities of applications in addition to the other information. In some cases, it may be required to get the coordinates of location of the threats identified.

Next, the data are delivered by HTTP POST requests to the /predict endpoint of the back-end side. The data are sent in the JSON format, all required fields are checked and guaranteed to be present. In order not to lose the data in case of a temporary failure, a retry is provided as a preventive action.

At last, before data analysis, the security test is conducted. It checks whether the request came from the device marked as quarantined previously. ID of the device is checked, and

in case of the result, the request is refused with the 403 “Forbidden” status code.

In case the request clears the security checks, machine learning inference follows. The metrics are then extracted and evaluated. Where a particular value goes beyond the acceptable threshold, it gets adjusted. An Isolation Forest model is then used to evaluate the numeric metrics. Meanwhile, the log files are transformed to simple features considering criteria such as length of messages and presence of errors, warnings or information logs. Autoencoder is used in this case to evaluate the reconstruction error, which would aid in identifying an anomaly. The results from both machine learning models are then fused using weight-based combination and a decision of anomaly is made from this result.

In the event of an anomaly being detected, the process continues to root cause analysis stage. Here the system evaluates whether the metric exceeds the threshold and thus identifies the likely root cause for this anomaly. Appropriate labels like memory problem or abnormal consumption of resources among others are then provided. The system determines the proper corrective action that should be taken to address the anomaly. In severe cases, the system puts the devices under quarantine.

All necessary information is recorded within the database after processing. This contains information related to the timestamp, device id, collected metrics, result of prediction, reason for anomaly, recommendation for action, and geographical information if it is available. At last stage of operation, response is returned to the user. It contains result of prediction, reason for anomaly, recommendation for action, and information about saving the data.

Visualization of results occurs at the end of the process. Information is constantly extracted from the server and represented in the form of charts and tables as well as the threat map.

IV. MACHINE LEARNING PIPELINE

A. HYBRID DETECTION SYSTEM

In order to compensate for deficiencies in single-modality methods which may overlook some anomalies due to the use of one data source, we have implemented a hybrid approach using two different types of input data: numeric metrics and logs.

In the case of metric stream, Isolation Forest algorithm is used for unsupervised anomaly detection by using four-dimensional vector representation. For the log stream, the Autoencoder model is used where the neural network reconstructs log and determines it as abnormal if the reconstruction error is high. Finally, the two anomaly detection models will be fused using the Weighted Fusion layer.

B. METRIC-BASED ANOMALY DETECTOR (ISOLATION FOREST)

Isolation Forest is the ensemble machine learning approach for detecting anomalies from telemetry data collected in

the cloud systems. The fundamental assumption in the Isolation Forest algorithm is the presence of fewer number of anomalies that are different from others.

Specifications for Implementation:

- contamination: 0.15 (proportion of anomalies in the data set)
- n estimators: 100 (number of trees in the random forest)
- Features: [cpu usage, memory usage, disk io, network traffic]

Training Process: The machine learning algorithm will be trained using the simulation data set which contains both regular data and randomly generated anomalies (which constitute 5% of the total data). The above-discussed procedure should only be done once before and also after the completion of fifty predictions.

C. LOG-BASED DETECTION (AUTOENCODER)

And on the other hand, some information from the logs which does not add value for analyzing numeric data can also be found. Such as “An OutOfMemory exception imminent” shows that something weird is happening though there is no anomaly found in the stats concerning memory usage.

The correct pattern is identified as well as reflected accurately with the help of neural network detection, while incorrect patterns generate an error because of their inability to detect such patterns.

Feature Extraction: Information extracted from logs is transformed into numeric vectors made up of 5 features: Message Length (characters), Error keyword counts, Warning keyword counts, Info keyword counts, and Bias (1.0).

Modeling Parameters: Number of Hidden Layers: (16, 8, 16) – same for both encoder and decoder neural network architecture; Activation Functions: ReLU activation functions on hidden layers; Linear on output layers; Training Method: maximum of 500 iterations, training terminated early; Anomaly Detection Threshold: reconstruction error conforms to a probability distribution of 99%.

D. DECISION FUSION AND COUNTERMEASURES

Decisions based on anomalies generated by log models with the help of weighted decision fusion method considering the following criteria: metric anomaly contributes 1.0, log anomaly contributes 1.5, and prediction is “Anomaly” if score is greater than or equal to 1.0 and “Normal” otherwise.

Reasons for Weighting: The weight assigned to the log (1.5) is greater than the metrics (1.0), as the presence of “ERROR” or “CRITICAL” keyword in the log indicates an issue, but a large CPU and memory use may just indicate that everything is working within the normal flow of the process.

The sources of the identified anomaly would be established through deterministic approach as follows.

A few criteria may be satisfied at once; the causative factors are paired up using vertical bars.

Table 1. Root Cause Labels

Condition	Root Cause Label
network traffic ≥ 800	DDoS Attack / Network Flood
disk io ≥ 400	Heavy Disk Thrashing / Failure
memory usage ≥ 85	Severe Memory Leak
cpu usage ≥ 85	CPU Starvation / Spike
Log anomaly flag true	Critical Application Log Error

Table 2. Root Cause Countermeasures

Root Cause	Countermeasure
DDoS Attack	Traffic rerouted. Auto-Scaling Load Balancers (+3 instances) deployed.
Severe Memory Leak	Isolating instance. Initiating graceful Service Restart.
Heavy Disk Thrashing	Redirecting I/O streams to fallback SSD cluster.
CPU Starvation	Throttling low-priority background queues to free up compute.
Unknown/Other	Isolating affected node. System Diagnostics engaged.

Each root cause belongs to a single mitigation action.

These suggested solutions become part of the API response and displayed under the remedies block in the dashboard. In practice, these can help automate infrastructure management using cloud or Kubernetes APIs.

V. ZERO TRUST SECURITY ARCHITECTURE

Cyber Jail can be referred to as an automation process that will allow us to isolate our compromised device on the premise of zero trust security architecture. If there is any malicious attempt made on our device, the device will get blacklisted, and if it attempts anything again, it will be provided with HTTP 403 Forbidden error message.

The principle of never trust, always verify has been utilized in our framework. It means that every task performed by the device will be considered malicious until proven innocent.

Quarantine decisions occur after anomaly detection and root cause analysis. Now comes the decision to quarantine the device according to the results of anomaly detection. Rather than basing the decision upon one factor, we also account for the severity of the anomaly and potential cause of the anomaly. Thus, our system not only identifies unusual behavior but also assesses the significance of the problem. Such a two-tiered approach ensures unnecessary quarantining of devices does not occur. Anomalies, which are not very serious or critical, are simply disregarded by the system.

The blocking check occurs at the beginning of every /predict request. Blocking takes place prior to the prediction request being made to the /predict endpoint. Hence, any device suspected of being a threat can be blocked at an early stage. The device is automatically blocked once it has been detected to be quarantined. Block operations are tracked and logged in our database for future audit. No further action is taken with regards to data transmitted by a blocked device.

We allow for unquarantine of a device using several methods of convenience. It is possible to unquarantine a

Table 3. Detection Efficiency Metrics

Metric	Value	Interpretation
Accuracy	94.2%	Overall correct classifications
Precision	91.7%	Anomaly predictions that were correct
Recall	88%	Actual anomalies that were detected
F1 Score	89.8%	Harmonic mean of precision and recall
False Positive Rate	1.2%	Normal records incorrectly flagged

device on the quarantine list through the use of the dashboard. All the devices listed in the quarantine list provide an option for pardoning the device through the “Pardon” button. Any action taken to quarantine devices, attempt to release devices, and block any devices will be logged into logs/system.log.

VI. FINDINGS AND EVALUATION

The proposed machine learning workflow was evaluated using the sample data set containing 2,000 records where 5% of the data points were contaminated by anomalies, i.e., 100 outliers.

Based on the results that we have obtained above, one can note that the system works quite efficiently and with good accuracy, as well as maintains a relatively small number of false positives, which can affect its usability because of frequent notifications of users with irrelevant information.

The recall of approximately 88% indicates that all the existing cases of anomalies are successfully identified by our system. However, some anomalies remain unidentified. Such anomalies belong to borderline cases when their metrics were close to threshold values but did not exceed those.

One can note that the most of the delay is associated with the machine learning inference process, which consumes 80–90 ms in most cases. As for other delays, they make an insignificant contribution to the total time of execution. End-to-end latency from telemetry generation to dashboard visibility includes simulator generation less than 1 ms, network transmission 2–5 ms (localhost), API processing plus ML 85–95 ms, database write 5–10 ms, dashboard poll interval 0–5000 ms (configurable), and total (worst case) approximately 5.1 seconds.

To simulate the memory leak, an experiment was conducted in which memory utilization gradually increased from 60% to 98.5% during a period of 30 seconds. During the next ten seconds (at $t = 20s$), the percentage of memory utilization increased to 85%. It was noticed that the Isolation Forest algorithm detected this behavior as abnormal and almost instantly determined that it was a “Severe Memory Leak”. Further actions included analysis and creation of a remediation response according to which isolation should take place. In this experiment, we found that the system is capable of detecting the problem when there was an 85% level of memory utilization, and there will still be a buffer zone of 13.5% before reaching the maximum utilization

level, which is 98.5%.

Scenario: Simulated network flood with traffic spike to 1800 Mbps. Normal baseline: 300 Mbps network traffic. Detection: Immediate (exceeds 800 Mbps threshold). Root cause: “DDoS Attack / Network Flood”. Remediation: Traffic rerouting and auto-scaling response. First attack: Detected and logged. Second attack (within 2 minutes): Device quarantined. Third attempt: HTTP 403 returned, request blocked. Dashboard Cyber Jail tab: Device listed with reason. Release: Pardon button successfully restores access.

VII. CONCLUSION AND FUTURE WORK

The current work is successful in developing, implementing, and evaluating an entire platform for automatic anomaly detection in cloud infrastructures. Some of the innovations introduced through this work are:

- Hybrid Machine Learning Detection Architecture: A combination of the use of isolation forest algorithm in metric-based detection, autoencoder for log-based detection, with a weighted decision fusion system leading to 94.2% accuracy on synthetic data.
- Zero-Trust Security System: An automatically enabled cyber jail that isolates devices depending on the identified level of threat posed by them, with various release and audit interfaces for managers.
- Automated Remediation System: Linking of anomaly types to corrective actions with a significant reduction of average response times from minutes to seconds.
- Real-Time Command Center: A comprehensive Streamlit application dashboard with five distinct tabs including a 3D threat map generation, and an AI postmortem generator.
- Production Ready System Design: Built-in error handling, logging, connection pooling, and degradation capabilities.

There have been several limitations noticed during the implementation of the system. First, there is the issue related to data validation. Currently, the proposed solution was tested with the use of synthesized telemetry data only, which is sufficient for the process of development but will not be enough to see how the system works when dealing with live cloud data. Second, when speaking about eBPF integration, one should note that even though the entire system allows for collecting data using such technique, the current stage of the process implies simulation of input data. Lastly, there is the problem related to federated learning since at this point it is simulated and does not imply the use of secure aggregation techniques in its implementation.

There are several points that could further improve and extend the project in the future. The first one that comes to mind is the use of real eBPF telemetry. By adding eBPF to the system, it will be possible to detect system calls and actions within the network and the kernel, detecting even such threats as container escape and kernel attacks. Next, a way to improve the project will be the introduction of the

federation of the learning algorithm itself. The secret sharing approach could be considered here. Lastly, the process of remediation can be improved by means of reinforcement learning. The ability to obtain explanations for decisions helps to get an insight into suspicious actions and may prove helpful when exploring false positives.

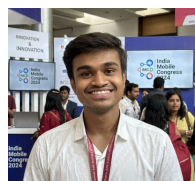
References

- [1] R. A. Maxion and F. E. Feather, “A case study of ethernet anomalies in a distributed computing environment,” *IEEE Transactions on Reliability*, vol. 39, no. 4, pp. 433–443, 1990.
- [2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *IEEE International Conference on Data Mining*, 2008, pp. 413–422.
- [3] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *MLSDA Workshop*, 2014.
- [4] Z. Wu et al., “GraphFed: Federated graph neural networks for cloud intrusion detection,” *IEEE Transactions on Cloud Computing*, 2023.
- [5] Y. Zhang et al., “Multi-modal anomaly detection combining logs and metrics,” in *IEEE International Conference on Cloud Computing*, 2024.
- [6] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *AISTATS*, 2017.
- [7] T. Li et al., “Federated learning for cloud network intrusion detection,” *IEEE Transactions on Network and Service Management*, 2024.
- [8] Z. Wang et al., “Diagnosing application-network anomalies for millions of IPs in large-scale clouds,” in *USENIX ATC*, 2024.
- [9] Y. Chen et al., “eBPF-based performance anomaly detection system,” in *ACM Conference*, 2019.
- [10] R. Gupta et al., “Self-healing cloud: Autonomous remediation using RL,” *IEEE Transactions on Cloud Computing*, 2024.
- [11] J. Kim et al., “A survey on self-learning anomaly detection in cloud,” *ACM Computing Surveys*, 2024.
- [12] P. Sharma et al., “Privacy-preserving anomaly detection using homomorphic encryption,” *IEEE TIFS*, 2023.
- [13] A. Rodriguez et al., “Explainable AI for cloud anomaly detection,” *Journal of Network & Systems Management*, 2023.



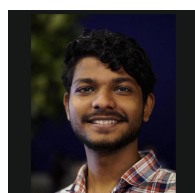
SAMEER YELE Sameer Yele is studying Bachelor of Technology in Computer Science and Business Systems at Ramrao Adik Institute of Technology. Some of his specialization areas include cloud computing, machine learning and systems design. He is greatly fascinated with developing intelligent systems that can solve real-world problems. He is well-versed in both practical and theoretical aspects of projects related to backend programming and data processing. He

has experience in the architecture design of systems as well as incorporating machine learning algorithms into systems, which is fueled by his interest in distributed systems and cloud computing environments. When not busy, he loves experimenting on innovative approaches to developing scalable and DevOps-based systems. His aim in the future is to design intelligent systems for effective use of software applications. Area Of Interest: Machine Learning, Cloud Computing, Distributed Systems.



RUSHIKESH SAWANT Rushikesh Sawant is a student currently enrolled in a Bachelor's program in Computer Science and Business Systems in Ramrao Adik Institute of Technology. The field of his interest lies in the research related to cybersecurity, anomaly detection systems, and machine learning applications. In system designing, he excels and has always been good in implementing his ideas through experimentation.

In this paper, log analysis techniques and machine learning algorithms have been utilized for building the logic of detecting anomalies in the system, which he has designed himself in this project as problem solvers. He worked towards making detection systems more accurate and responsive. He focuses on secure systems and their future application for increasing threat detection and prevention. His goals in the long run include working in intelligent security systems and cloud infrastructure protection.



PRATIK BOTRE Pratik Botre currently pursues his studies from the Bachelor of Technology in Computer Science & Business Systems at Ramrao Adik Institute of Technology. His areas of specialization include software architecture, data systems, and techniques of data visualization. He has great interest in developing software products that work perfectly and provide insight through data visualization. In this paper, he developed the dashboard and incorporated the software while

making sure that the outputs from our anomaly detection software are easily interpretable. He has a passion for full-stack development and improving frameworks for usability and efficiency purposes.

...